

FinGuard AI — Unified Banking Risk & Compliance Intelligence Platform

One project. Every AI/ML engineering concept. Built in the order you learn it.

Why this project

A bank/NBFC needs three things AI can do, plus one thing that ties them together:

- 1. Decide who gets a loan (and explain why) — **classical ML**
- 2. Catch fraudulent transactions and fake KYC documents in real time — **deep learning**
- 3. Let compliance/ops staff ask questions in plain English and get grounded answers — **GenAI/LLM**
- 4. Run all of this reliably in production — **MLOps**

This isn't a contrived combination — real fintech ML platforms are built exactly this way, with multiple model services sitting behind one deployment layer. It also lines up directly with why Mumbai pays a fintech premium for fraud/risk ML, and why Pune's GCC banks (JPMorgan, Barclays, Deutsche Bank-type teams) hire for this exact skill combination.

Datasets (all free/public — no data collection needed)

Module	Dataset
Credit risk	Kaggle "Give Me Some Credit" or Home Credit Default Risk, or German Credit Data (UCI)
Fraud detection	Kaggle Credit Card Fraud Detection (highly imbalanced, real transaction features)
Document intelligence	RVL-CDIP (document classification) + synthetic ID/cheque images you generate, or a driving-license/ID-card dataset from Kaggle
GenAI knowledge base	RBI guideline PDFs (public), your own written bank policy/FAQ docs, loan T&Cs

Tech stack (the "most demanding" stack, applied)

Python · Pandas/NumPy · Scikit-learn · XGBoost · SHAP · PyTorch · OpenCV/Tesseract · LangChain or LlamaIndex · Hugging Face · FAISS/Chroma · FastAPI · Docker · Kubernetes · MLflow · GitHub Actions · Evidently AI · AWS or GCP (pick one)

Phase 0 — Foundation & Architecture (~1 week)

Do this before touching any model code.

- ☐ Write a one-page problem statement: what does FinGuard AI do, for whom, and why
- ☐ Set up the repo: `data/`, `notebooks/`, `src/`, `services/`, `models/`, `tests/`, `docker/`, `.github/workflows/`
- ☐ Set up a Python virtual environment + `requirements.txt` per module (keep classical ML, DL, and GenAI deps separate — they'll conflict otherwise)
- ☐ Install and configure **MLflow** now, even before you have a model — every phase from here on logs into it. This habit alone is a strong interview signal
- ☐ Write the architecture diagram (the one you just saw) into your README
- ☐ Set up git with a real branching convention (`feature/credit-risk-model`, etc.) — recruiters do look at commit history

Checkpoint: empty-but-structured repo, MLflow running locally, README with the architecture diagram and problem statement.

Phase 1 — Classical ML Module: Credit Risk Scoring Engine

You already know this material, so move quickly here — 1-2 weeks. This becomes "Model Service 1."

Build, in order:

1. EDA on the credit dataset — missingness, target imbalance, correlations
2. Feature engineering: debt-to-income ratios, encoding categoricals, handling missing values properly (not just mean-fill — justify your choice)
3. Handle class imbalance: SMOTE or class weighting (state which you chose and why in your notes — interviewers ask this)
4. Train and compare: Logistic Regression (baseline) → Random Forest → XGBoost
5. Evaluate properly: since this is imbalanced, use ROC-AUC and Precision-Recall curves, **not** plain accuracy
6. Hyperparameter tuning: Optuna or GridSearchCV on your best model
7. **Explainability with SHAP** — this is the single highest-value addition for a fintech project, because real credit models are legally required to be explainable. Generate SHAP plots for individual rejected applications
8. Log every run (params, metrics, artifacts) to MLflow
9. Save the final model + preprocessing pipeline as a single artifact (this is what Phase 4 will containerize)

Checkpoint: a trained, explainable credit-scoring model, versioned in MLflow, with a notebook or script that takes a new applicant's data and returns a score + a SHAP explanation.

Phase 2 — Deep Learning Module (your current focus)

Two sub-modules — build both, they exercise different DL skills.

2A. Transaction Fraud Detection

1. Load the fraud dataset, note the severe imbalance (fraud is usually <1% of transactions)
2. Build an **autoencoder** (unsupervised): train only on normal transactions, flag high reconstruction-error transactions as anomalies
3. Build a **supervised deep classifier** (feedforward net, or an LSTM if you frame transactions as per-user sequences) using a weighted or focal loss to handle imbalance
4. Compare both DL approaches against your Phase 1 classical model on the same test set — same metrics (ROC-AUC, PR-AUC), plus inference latency
5. Write up the comparison: which approach caught more fraud, which was faster, what the trade-off was. **This comparison, in your own words, is one of the best things you can say in an interview** — it shows you understand *when* DL is worth its cost, not just that you can use it
6. Log everything to MLflow (PyTorch supports MLflow autologging)

2B. KYC Document Intelligence

1. Build a CNN-based classifier: is this a valid ID document, and which type (PAN card, Aadhaar, passport, driving license)?
2. Use transfer learning — fine-tune a pretrained ResNet or EfficientNet rather than training from scratch
3. Add OCR (Tesseract or EasyOCR) to extract fields (name, ID number, date of birth) from the validated document
4. Add a simple rule-based or ML-based validation layer on the extracted fields (does the ID number match the expected format? Is the date of birth plausible?)
5. Log training runs, save the fine-tuned model

Checkpoint: two working deep learning services — one flags fraudulent transactions, one verifies and extracts data from ID documents — both benchmarked and logged in MLflow. This is "Model Service 2" and "Model Service 3."

Phase 3 — GenAI/LLM Layer

Build this once you've studied embeddings, RAG, and agents.

1. **Build the knowledge base:** collect your bank policy/FAQ/RBI guideline documents, chunk them sensibly (by section, not arbitrary character counts), embed with a sentence-transformer model
2. Store embeddings in a vector DB — FAISS or Chroma locally (no cost, good enough for a portfolio project)
3. Build the **RAG pipeline** with LangChain or LlamaIndex: retrieve relevant chunks, pass to an LLM (an API like Claude/GPT, or an open model via Hugging Face if you want a fully open-source stack), generate a grounded answer

4. **Add the agent/tool-calling layer** — this is what makes it a "compliance copilot" instead of just a chatbot:
 - Tool 1: "check applicant X's credit risk score" → calls your Phase 1 model service
 - Tool 2: "explain why transaction Y was flagged" → calls your Phase 2A model + returns the SHAP-style explanation
 - Tool 3: "verify this uploaded document" → calls your Phase 2B service
5. **Optional stretch (do this if you want the ₹25L+ tier of differentiation):** fine-tune a small open model with LoRA to generate standardized risk-explanation reports in a consistent tone/format
6. Evaluate your RAG system — don't skip this. Build a small eval set of question/expected-answer pairs and check retrieval quality, not just "it looks right when I try it"

Checkpoint: an AI assistant that can answer policy questions grounded in real documents, and can call your other three models as tools when asked things like "why was this loan rejected?"

Phase 4 — MLOps & Deployment

Build this once you've studied Docker, Kubernetes, and CI/CD. This is what unifies everything above into one product.

1. **Containerize each service independently:** `credit-risk-api`, `fraud-detection-api`, `document-verification-api`, `genai-assistant-api` — each its own Dockerfile
2. **Serve each with FastAPI:** clean REST endpoints, input validation with Pydantic, proper error handling
3. **Orchestrate:** start with `docker-compose` locally, then move to Kubernetes (minikube/kind is fine for learning — you don't need a cloud cluster to prove the skill)
4. **MLflow Model Registry:** register all four models with versions, stages (staging/production), so you can demonstrate a real promotion workflow
5. **CI/CD with GitHub Actions:** on push — run tests, lint, build Docker images, (optionally) deploy to a free-tier cloud instance
6. **Monitoring:** set up Evidently AI (or a custom dashboard) to track data drift and prediction drift — genuinely important here, since transaction and applicant patterns shift over time, and this is the exact skill gap the earlier research flagged as most in-demand and hardest to fill
7. **Pick one cloud** (AWS or GCP) and deploy at least the gateway + one service there, even on a free tier — a live link matters more than perfect infra
8. **Build a simple unified dashboard** (Streamlit is fastest) that ties all four services into one visible product: submit a loan application → see the credit score + SHAP explanation; upload a transaction → see fraud flag; upload a document → see verification result; ask the assistant a question → see a grounded answer that can reference the other models

Checkpoint: four containerized services, orchestrated together, monitored for drift, deployed somewhere reachable by a URL, all wrapped in one dashboard.

Phase 5 — Integration, Testing & Portfolio Packaging (~1-2 weeks)

- ☐ End-to-end test: run a full scenario through all four services in sequence
 - ☐ Basic load testing with Locust — even simple numbers ("handles X requests/sec") are a good talking point
 - ☐ Write a genuinely good README: architecture diagram, problem statement, tech decisions and *why* you made them, how to run it locally
 - ☐ Record a 3–5 minute demo video walking through the dashboard
 - ☐ Write 2–3 short technical posts on specific hard parts — e.g. "Classical ML vs deep learning for imbalanced fraud detection: what I found" or "Why I chose SHAP over LIME for credit explainability"
 - ☐ Make sure the live deployed link actually works before you put it on your resume
-

Concept coverage checklist (nothing left out)

Classical ML: regression, classification, ensembles (RF/XGBoost), imbalanced data handling, feature engineering, cross-validation, hyperparameter tuning, model explainability (SHAP)

Deep Learning: CNNs, transfer learning, autoencoders (unsupervised anomaly detection), sequence models (LSTM), custom/weighted loss functions, PyTorch training loops, OCR integration

GenAI/LLM: embeddings, vector databases, RAG, prompt engineering, agents/tool-calling, LLM evaluation, (optional) LoRA fine-tuning

MLOps: Docker, Kubernetes, FastAPI, MLflow (tracking + registry), CI/CD (GitHub Actions), drift monitoring, cloud deployment

Engineering fundamentals throughout: system design, API design, testing, version control discipline, load testing

Realistic timeline

Phase	Time	Depends on you having studied
0 — Foundation	~1 week	Basic Python, git
1 — Classical ML	1-2 weeks	Classical ML (already done)
2 — Deep Learning	3-4 weeks	Deep Learning (your current stage)
3 — GenAI/LLM	3-4 weeks	Embeddings, RAG, LLM APIs
4 — MLOps	3-4 weeks	Docker, K8s basics, CI/CD
5 — Integration & polish	1-2 weeks	—

Total: ~4-5 months, running in parallel with the topic study itself rather than after it — you build each phase right after you finish studying its topic, which is exactly the plan you described.